

Lazy Cost Analysis is Functional Logic Programming

First Author^{1,2*}, Second Author^{2,3†} and Third Author^{1,2†}

¹*Department, Organization, Street, City, 100190, State, Country.

²Department, Organization, Street, City, 10587, State, Country.

³Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): iauthor@gmail.com;
Contributing authors: iauthor@gmail.com; iiiauthor@gmail.com;

[†]These authors contributed equally to this work.

Abstract

The abstract serves both as a general introduction to the topic and as a brief, non-technical summary of the main results and their implications. Authors are advised to check the author instructions for the journal they are submitting to for word limits and if structural elements like subheadings, citations, or equations are permitted.

Keywords: keyword1, Keyword2, Keyword3, Keyword4

1 Introduction

1.1 what is lazy evaluation?

- what is lazy evaluation?
- why is it cool?
- talk about cost analysis specifically
- (maybe) briefly explain the standard semantics for laziness and its advantages/disadvantages
- briefly explain clairvoyance and demand semantics
- talk about our troubles with writing demand terms
- explain the solution
- list of contributions

2 Motivating example(s)

- a more in-depth discussion of lazy evaluation semantics, esp. clairvoyance and demand semantics
- give an example that models each semantics in some programming language (probably Curry)
 - this will require displaying/explaining the basic infrastructure
 - what example? should be fairly small. take-append?
- discuss how this is very annoying to write in demand semantics
- show how, in a logic language, you can “write both in one program”
- this will need a discussion of what logic programming is, and probably a brief explanation of the specific language we are using (probably Curry)

3 Technical stuff

4 The base language

4.1 Syntax

```

      A ::= Bool
          | List A
          | Thunk A
t ::= x
    | let x = t in t
    | true
    | false
    | if t then t else t
    | []
    | t :: t
    | foldr( $\lambda xx.t$ , t, t)
    | lazy t
    | force t
    | tick t
```

The base language is a simple calculus with just enough types to make things interesting. The explicit **lazy force** terms decouple our analysis from any particular evaluation model, and the explicit **tick** terms decouple it from any particular cost model.

Note that the language does not have functions. This is due to a technical limitation of demand semantics.

4.2 Typing

$$\frac{\Gamma(x) = A}{\Gamma \vdash^{\mathbf{B}} x : A}$$

$$\begin{array}{c}
\frac{\Gamma \vdash^{\mathbf{B}} t_1 : A \quad \Gamma, x : A \vdash^{\mathbf{B}} t_2 : B}{\Gamma \vdash^{\mathbf{B}} \text{let } x = t_1 \text{ in } t_2 : B} \\
\frac{}{\Gamma \vdash^{\mathbf{B}} \text{true} : \text{Bool}} \\
\frac{}{\Gamma \vdash^{\mathbf{B}} \text{false} : \text{Bool}} \\
\frac{\Gamma \vdash^{\mathbf{B}} t_1 : \text{Bool} \quad \Gamma \vdash^{\mathbf{B}} t_2 : A \quad \Gamma \vdash^{\mathbf{B}} t_3 : A}{\Gamma \vdash^{\mathbf{B}} \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : A} \\
\frac{\Gamma \vdash^{\mathbf{B}} [] : \text{List } A \quad \Gamma \vdash^{\mathbf{B}} t_2 : \text{Thunk } (\text{List } A)}{\Gamma \vdash^{\mathbf{B}} t_1 : A \quad \Gamma \vdash^{\mathbf{B}} t_2 : \text{Thunk } (\text{List } A)} \\
\frac{\Gamma \vdash^{\mathbf{B}} t_1 :: t_2 : \text{List } A \quad \Gamma, x_1 : A, x_2 : \text{Thunk } B \vdash^{\mathbf{B}} t_1 : B \quad \Gamma \vdash^{\mathbf{B}} t_2 : B \quad \Gamma \vdash^{\mathbf{B}} t_3 : \text{List } A}{\Gamma \vdash^{\mathbf{B}} \text{foldr}(\lambda x_1 x_2. t_1, t_2, t_3) : B} \\
\frac{}{\Gamma \vdash^{\mathbf{B}} t : A} \\
\frac{\Gamma \vdash^{\mathbf{B}} \text{lazy } t : \text{Thunk } A \quad \Gamma \vdash^{\mathbf{B}} t : \text{Thunk } A}{\Gamma \vdash^{\mathbf{B}} \text{force } t : A} \\
\frac{}{\Gamma \vdash^{\mathbf{B}} t : A} \\
\frac{}{\Gamma \vdash^{\mathbf{B}} \text{tick } t : A}
\end{array}$$

4.3 Exact values and approximations

Each type A comes equipped with a set $\mathbb{E}[A]$ of *exact values* and a set $\mathbb{A}[A]$ of *approximation values*.

$$\begin{aligned}
\mathbb{E}[\text{Bool}] &= \{\text{true}, \text{false}\} \\
\mathbb{E}[\text{Thunk } A] &= \mathbb{E}[A] \\
\mathbb{E}[\text{List } A] &= \{\varepsilon\} \cup \{(v_1, v_2) \mid v_1 \in \mathbb{E}[A], v_2 \in \mathbb{E}[\text{List } A]\}
\end{aligned}$$

$$\begin{aligned}
\mathbb{A}[\text{Bool}] &= \{\text{true}, \text{false}\} \\
\mathbb{A}[\text{Thunk } A] &= \{\perp\} \cup \{\text{sthunkZ } v \mid v \in \mathbb{A}[A]\} \\
\mathbb{A}[\text{List } A] &= \{\varepsilon\} \cup \{(v_1, v_2) \mid v_1 \in \mathbb{A}[A], v_2 \in \mathbb{A}[\text{Thunk } (\text{List } A)]\}
\end{aligned}$$

4.4 Evaluation semantics

Evaluation semantics

$$\begin{aligned}
\mathbb{E}[x](\gamma) &= \gamma(x) \\
\mathbb{E}[\text{let } x = t_1 \text{ in } t_2](\gamma) &= \mathbb{E}[t_2](\gamma, x \mapsto \mathbb{E}[t_1](\gamma)) \\
\mathbb{E}[\text{true}](\gamma) &= \text{true}
\end{aligned}$$

$$\begin{aligned}
\mathbb{E}[\text{false}](\gamma) &= \text{false} \\
\mathbb{E}[\text{if } t_1 \text{ then } t_2 \text{ else } t_3](\gamma) &= \begin{cases} \mathbb{E}[t_2](\gamma) & \text{if } \mathbb{E}[t_1](\gamma) = \text{true} \\ \mathbb{E}[t_3](\gamma) & \text{otherwise} \end{cases} \\
\mathbb{E}[\square](\gamma) &= \varepsilon \\
\mathbb{E}[t_1 :: t_2](\gamma) &= (\mathbb{E}[t_1](\gamma), \mathbb{E}[t_2](\gamma)) \\
\mathbb{E}[\text{foldr}(\lambda x_1 x_2. t_1, t_2, t_3)](\gamma) &= \text{foldr}((v_1, v_2) \mapsto \mathbb{E}[t_1](\gamma, x_1 \mapsto v_1, x_2 \mapsto v_2), \mathbb{E}[t_2](\gamma), \mathbb{E}[t_3](\gamma)) \\
\mathbb{E}[\text{lazy } t](\gamma) &= \mathbb{E}[t](\gamma) \\
\mathbb{E}[\text{force } t](\gamma) &= \mathbb{E}[t](\gamma) \\
\mathbb{E}[\text{tick } t](\gamma) &= \mathbb{E}[t](\gamma)
\end{aligned}$$

$$\text{foldr}(f, y, (x_1, x_2, \dots, x_n)) = f(x_1, f(x_2, \dots f(x_n, y) \dots))$$

Lemma 1. *If $\Gamma \vdash^{\mathbf{B}} t : A$ and $\gamma \in \mathbb{E}[\Gamma]$, then $\mathbb{E}[t](\gamma) \in \mathbb{E}[A]$.*

4.5 Clairvoyance semantics

Clairvoyance semantics has the judgment schema $\gamma ; t \Downarrow_c^{\mathbb{C}} v$. Intuitively, c is the number of ticks encountered while evaluating t to v .

$$\begin{array}{c}
\frac{\gamma(x) = v}{\gamma ; x \Downarrow_0^{\mathbb{C}} v} \\
\frac{\gamma ; t_1 \Downarrow_{c_1}^{\mathbb{C}} v_1 \quad \gamma, x \mapsto v_1 ; t_2 \Downarrow_{c_2}^{\mathbb{C}} v_2}{\gamma ; \text{let } x = t_1 \text{ in } t_2 \Downarrow_{c_1+c_2}^{\mathbb{C}} v_2} \\
\frac{}{\gamma ; \text{true} \Downarrow_0^{\mathbb{C}} \text{true}} \\
\frac{\gamma ; \text{false} \Downarrow_0^{\mathbb{C}} \text{false}}{\gamma ; t_1 \Downarrow_{c_1}^{\mathbb{C}} \text{true} \quad \gamma ; t_2 \Downarrow_{c_2}^{\mathbb{C}} v} \\
\frac{\gamma ; \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow_{c_1+c_2}^{\mathbb{C}} v}{\gamma ; t_1 \Downarrow_{c_1}^{\mathbb{C}} \text{false} \quad \gamma ; t_3 \Downarrow_{c_3}^{\mathbb{C}} v} \\
\frac{}{\gamma ; \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow_{c_1+c_3}^{\mathbb{C}} v} \\
\frac{\gamma ; \square \Downarrow_0^{\mathbb{C}} \varepsilon}{\gamma ; t_1 \Downarrow_{c_1}^{\mathbb{C}} v_1 \quad \gamma ; t_2 \Downarrow_{c_2}^{\mathbb{C}} v_2} \\
\frac{\gamma ; t_1 :: t_2 \Downarrow_{c_1+c_2}^{\mathbb{C}} (v_1, v_2)}{\gamma ; t_3 \Downarrow_{c_1}^{\mathbb{C}} v_1 \quad \gamma ; \lambda x_1 x_2. t_1, t_2, v_1 \Downarrow_{c_2}^{\mathbb{C}\text{-foldr}} v_2} \text{ L-FOLDR} \\
\frac{}{\gamma ; \text{foldr}(\lambda x_1 x_2. t_1, t_2, t_3) \Downarrow_{c_1+c_2}^{\mathbb{C}} v_2} \\
\frac{\gamma ; t \Downarrow_c^{\mathbb{C}} v}{\gamma ; \text{lazy } t \Downarrow_c^{\mathbb{C}} \text{sthunkZ } v} \text{ L-LAZY-THUNK} \\
\frac{}{\gamma ; \text{lazy } t \Downarrow_0^{\mathbb{C}} \perp} \text{ L-LAZY-UNDEFINED}
\end{array}$$

$$\frac{\frac{\gamma ; t \Downarrow_c^{\mathbb{C}} \text{sthunkZ } v}{\gamma ; \text{force } t \Downarrow_c^{\mathbb{C}} v} \text{ L-FORCE}}{\gamma ; t \Downarrow_c^{\mathbb{C}} v} \quad \frac{\gamma ; \text{tick } t \Downarrow_{c+1}^{\mathbb{C}} v}{\gamma ; \text{tick } t \Downarrow_{c+1}^{\mathbb{C}} v}$$

The crucial rules are L-LAZY-THUNK, L-LAZY-UNDEFINED, L-FORCE. L-LAZY-THUNK and L-LAZY-UNDEFINED say that, to evaluate a **lazy** term, we nondeterministically either evaluate its subterm and wrap it in a **thunk** or substitute an “empty” value $\perp$. L-FORCE says that we can only evaluate a value of **Thunk** type if it is in fact a **thunk**; in other words, if we need the result of a **lazy** term, then we must have actually evaluated it via L-LAZY-THUNK.

In the original formulation of clairvoyance semantics, nondeterminism was introduced at **let** bindings: when evaluating **let** $x = t_1$ **in** t_2 , either x was immediately bound to the result of evaluating t_1 , or else x was not bound at all. But our base language explicitly marks laziness points with **thunk** terms, so we must introduce nondeterminism there.

Note that **foldr** is evaluated by first evaluating the list argument to a value and then iterating over that value. The iteration is expressed in the following auxiliary relation.

$$\frac{\frac{\gamma ; t_2 \Downarrow_c^{\mathbb{C}} v}{\gamma ; \lambda x_1 x_2. t_1, t_2, \varepsilon \Downarrow_c^{\mathbb{C}\text{-foldr}} v} \text{ C-FOLDER-NIL}}{\gamma, x_1 \mapsto v_1, x_2 \mapsto \perp ; t_1 \Downarrow_c^{\mathbb{C}} v} \text{ C-FOLDER-UNDEFINED} \quad \frac{\gamma ; \lambda x_1 x_2. t_1, t_2, (v_1, \perp) \Downarrow_c^{\mathbb{C}\text{-foldr}} v}{\gamma ; \lambda x_1 x_2. t_1, t_2, v_2 \Downarrow_{c_1}^{\mathbb{C}\text{-foldr}} v_3 \quad \gamma, x_1 \mapsto v_1, x_2 \mapsto \text{sthunkZ } v_3 ; t_1 \Downarrow_{c_2}^{\mathbb{C}} v} \text{ C-FOLDER-THUNK}$$

Lemma 2. *If $\Gamma \vdash^{\mathbf{B}} t : A$, $\gamma \in \mathbb{A}[\Gamma]$, and $\gamma ; t \Downarrow_c^{\mathbb{C}} v$, then $v \in \mathbb{A}[A]$.*

Lemma 3. *If $\Gamma \vdash^{\mathbf{B}} t : A$ and $\gamma \in \mathbb{A}[\Gamma]$, then $\gamma ; t \Downarrow_c^{\mathbb{C}}$.*

4.6 Demand semantics

$$\begin{aligned} \mathbb{D}[x](\gamma, a) &= (\{x \mapsto a\}, 0) \\ \mathbb{D}[\text{true}](\gamma, a) &= (\perp, 0) \\ \mathbb{D}[\text{false}](\gamma, a) &= (\perp, 0) \\ \mathbb{D}[\square](\gamma, a) &= (\perp, 0) \\ \mathbb{D}[t_1 :: t_2](\gamma, a) &= \mathbb{D}[t_1](\gamma, a) \sqcup \mathbb{D}[t_2](\gamma, a) \end{aligned}$$

5 The logic language

5.1 Syntax

$$\begin{array}{lcl}
 A & ::= & \text{Bool} \\
 & & | A * A \\
 & & | \text{Thunk } A \\
 & & | \text{Nat} \\
 & & | \text{List } A \\
 \\
 t & ::= & x \\
 & & | \text{let } x = t \text{ in } t \\
 & & | \text{true} \\
 & & | \text{false} \\
 & & | t == t \\
 & & | t <= t \\
 & & | \text{if } t \text{ then } t \text{ else } t \\
 & & | (t, t) \\
 & & | \text{thunk } t \\
 & & | \text{undefined} \\
 & & | \text{case } t \text{ of thunk } x \Rightarrow t; \text{undefined} \Rightarrow t \\
 & & | n \\
 & & | t + t \\
 & & | \text{if } t \text{ then } t \text{ else } t \\
 & & | [] \\
 & & | t :: t \\
 & & | \text{foldr}(\lambda x x.t, t, t) \\
 & & | t ? t \\
 & & | \text{free } A \\
 & & | \text{fail}
 \end{array}$$

5.2 Typing

$$\frac{\Gamma, x_1 : A, x_2 : \text{Thunk } B \vdash^{\mathbf{B}} t_1 : B \quad \Gamma \vdash^{\mathbf{B}} t_2 : B \quad \Gamma \vdash^{\mathbf{B}} t_3 : \text{List } A}{\Gamma \vdash^{\mathbf{B}} \text{foldr}(\lambda x_1 x_2.t_1, t_2, t_3) : B}$$

5.3 Semantics

$$\frac{\frac{\gamma(x) = v}{\gamma; x \Downarrow^{\mathbb{L}} v} \quad \gamma; t_1 \Downarrow^{\mathbb{L}} v_1 \quad \gamma, x \mapsto v_1; t_2 \Downarrow^{\mathbb{L}} v_2}{\gamma; \text{let } x = t_1 \text{ in } t_2 \Downarrow^{\mathbb{L}} v_2} \\
 \frac{}{\gamma; \text{true} \Downarrow^{\mathbb{L}} \text{true}} \\
 \frac{}{\gamma; \text{false} \Downarrow^{\mathbb{L}} \text{false}} \\
 \frac{\gamma; t_1 \Downarrow^{\mathbb{L}} v \quad \gamma; t_2 \Downarrow^{\mathbb{L}} v}{\gamma; t_1 == t_2 \Downarrow^{\mathbb{L}} \text{true}}$$

$$\begin{array}{c}
\frac{\gamma; t_1 \Downarrow^{\mathbb{L}} v_1 \quad \gamma; t_2 \Downarrow^{\mathbb{L}} v_2 \quad v_1 \neq v_2}{\gamma; t_1 \Downarrow^{\mathbb{L}} v_1 \quad \gamma; t_2 \Downarrow^{\mathbb{L}} v_2 \quad v_1 \leq v_2} \\
\frac{\gamma; t_1 \Downarrow^{\mathbb{L}} v_1 \quad \gamma; t_2 \Downarrow^{\mathbb{L}} v_2 \quad v_1 \not\leq v_2}{\gamma; t_1 \Downarrow^{\mathbb{L}} v_1 \quad \gamma; t_2 \Downarrow^{\mathbb{L}} v_2 \quad v_1 \leq v_2} \\
\frac{\gamma; t_1 \Downarrow^{\mathbb{L}} \text{true} \quad \gamma; t_2 \Downarrow^{\mathbb{L}} v}{\gamma; \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow^{\mathbb{L}} v} \\
\frac{\gamma; t_1 \Downarrow^{\mathbb{L}} \text{false} \quad \gamma; t_3 \Downarrow^{\mathbb{L}} v}{\gamma; \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow^{\mathbb{L}} v} \\
\frac{\gamma; t_1 \Downarrow^{\mathbb{L}} v_1 \quad \gamma; t_2 \Downarrow^{\mathbb{L}} v_2}{\gamma; (t_1, t_2) \Downarrow^{\mathbb{L}} (v_1, v_2)} \\
\frac{\gamma; t \Downarrow^{\mathbb{L}} v}{\gamma; \text{thunk } t \Downarrow^{\mathbb{L}} \text{sthunkZ } v} \\
\frac{\gamma; \text{undefined} \Downarrow^{\mathbb{L}} \perp}{\gamma; t_1 \Downarrow^{\mathbb{L}} \text{sthunkZ } v_1 \quad \gamma, x \mapsto v_1; t_2 \Downarrow^{\mathbb{L}} v_2} \\
\frac{\gamma; \text{case } t_1 \text{ of thunk } x \Rightarrow t_2; \text{undefined} \Rightarrow t_3 \Downarrow^{\mathbb{L}} v_2}{\gamma; t_1 \Downarrow^{\mathbb{L}} \perp \quad \gamma; t_3 \Downarrow^{\mathbb{L}} v} \\
\frac{\gamma; \text{case } t_1 \text{ of thunk } x \Rightarrow t_2; \text{undefined} \Rightarrow t_3 \Downarrow^{\mathbb{L}} v}{\gamma; n \Downarrow^{\mathbb{L}} n} \\
\frac{\gamma; t_1 \Downarrow^{\mathbb{L}} n_1 \quad \gamma; t_2 \Downarrow^{\mathbb{L}} n_2}{\gamma; t_1 + t_2 \Downarrow^{\mathbb{L}} n_1 + n_2} \\
\frac{\gamma; [] \Downarrow^{\mathbb{L}} \varepsilon}{\gamma; t_1 \Downarrow^{\mathbb{L}} v_1 \quad \gamma; t_2 \Downarrow^{\mathbb{L}} v_2} \\
\frac{\gamma; t_1 :: t_2 \Downarrow^{\mathbb{L}} (v_1, v_2)}{\gamma; t_3 \Downarrow^{\mathbb{L}} v_1 \quad \gamma; \lambda x_1 x_2. t_1, t_2, v_1 \Downarrow^{\mathbb{L}\text{-foldr}} v_2} \\
\frac{\gamma; \text{foldr}(\lambda x_1 x_2. t_1, t_2, t_3) \Downarrow^{\mathbb{L}} v_2}{v : A} \\
\frac{\gamma; \text{free } A \Downarrow^{\mathbb{L}} v}{\gamma; t_1 \Downarrow^{\mathbb{L}} v} \\
\frac{\gamma; t_1 ? t_2 \Downarrow^{\mathbb{L}} v}{\gamma; t_2 \Downarrow^{\mathbb{L}} v} \\
\frac{\gamma; t_1 ? t_2 \Downarrow^{\mathbb{L}} v}{\gamma; t_1 ? t_2 \Downarrow^{\mathbb{L}} v}
\end{array}$$

As in clairvoyance semantics, the semantics of **foldr** is defined in terms of an auxiliary relation.

$$\frac{\gamma; t_2 \Downarrow^{\mathbb{L}} v}{\gamma; \lambda x_1 x_2. t_1, t_2, \varepsilon \Downarrow^{\mathbb{L}\text{-foldr}} v}$$

$$\frac{\gamma, x_1 \mapsto v_1, x_2 \mapsto \perp; t_1 \Downarrow^{\mathbb{L}} v}{\gamma; \lambda x_1 x_2. t_1, t_2, (v_1, \perp) \Downarrow^{\mathbb{L}\text{-foldr}} v} \\
\frac{\gamma; \lambda x_1 x_2. t_1, t_2, \text{sthunkZ } v_2 \Downarrow^{\mathbb{L}\text{-foldr}} v_3 \quad \gamma, x_1 \mapsto v_1, x_2 \mapsto \text{sthunkZ } v_3; t_1 \Downarrow^{\mathbb{L}} v}{\gamma; \lambda x_1 x_2. t_1, t_2, (v_1, v_2) \Downarrow^{\mathbb{L}\text{-foldr}} v}$$

Lemma 4. *If $\Gamma \vdash^{\mathbf{L}} t : A$, $\gamma \in \mathbb{A}[\Gamma]$, and $\gamma; t \Downarrow^{\mathbb{L}} v$, then $v \in \mathbb{A}[A]$.*

Note that **fail** never reduces, so the language is not total.

5.4 The clairvoyance translation

We now define some constructs on the terms in **L**. For each construct c , we provide typing rules and a set of evaluation rules that *completely characterize* c . By “completely characterize”, we mean the following.

- *Soundness:* Each rule is valid.
- *Completeness:* Any derivation of the form $\gamma; c \Downarrow^{\mathbb{L}} v$ must have the form of one of the elimination rules for that construct. In particular, there must be some elimination rule whose hypotheses are all true. This allows us to “invert” judgments of the form $\gamma; c \Downarrow^{\mathbb{L}} v$.

5.4.1 return

$$\frac{\text{return } t = (t, 0) \quad \Gamma \vdash^{\mathbf{L}} t : A}{\Gamma \vdash^{\mathbf{L}} \text{return } t : \mathbb{M} A} \\
\frac{\Gamma \vdash^{\mathbf{L}} \text{return } t : \mathbb{M} A \quad \gamma; t \Downarrow^{\mathbb{L}} v}{\gamma; \text{return } t \Downarrow^{\mathbb{L}} (v, 0)}$$

5.4.2 bind

$$\text{do } x \leftarrow t_1; t_2 = \text{let } (x, y_1) = t_1 \text{ in let } (y_2, y_3) = t_2 \text{ in } (y_2, y_1 + y_3) \\
\frac{\Gamma \vdash^{\mathbf{L}} t_1 : \mathbb{M} A \quad \Gamma, x : A \vdash^{\mathbf{L}} t_2 : \mathbb{M} B}{\Gamma \vdash^{\mathbf{L}} \text{do } x \leftarrow t_1; t_2 : \mathbb{M} B}$$

5.4.3 transpose

transpose $t = \text{case } t \text{ of thunk } y_1 \rightarrow \text{do } y_2 \leftarrow y_1; \text{return } (\text{thunk } y_2)$
undefined $\rightarrow \text{return undefined}$

$$\frac{\Gamma \vdash^{\mathbf{L}} t : \text{Thunk } (\mathbb{M} A)}{\Gamma \vdash^{\mathbf{L}} \text{transpose } t : \mathbb{M} (\text{Thunk } A)} \\
\frac{\Gamma \vdash^{\mathbf{L}} \text{transpose } t : \mathbb{M} (\text{Thunk } A) \quad \gamma; t \Downarrow^{\mathbb{L}} \perp}{\gamma; \text{transpose } t \Downarrow^{\mathbb{L}} (\perp, 0)} \\
\frac{\gamma; \text{transpose } t \Downarrow^{\mathbb{L}} (\perp, 0) \quad \gamma; t \Downarrow^{\mathbb{L}} \text{sthunkZ } (v, c)}{\gamma; \text{transpose } t \Downarrow^{\mathbb{L}} (\text{sthunkZ } v, c)}$$

5.4.4 foldM

$\text{foldM}(\lambda x_1 x_2. t_1, t_2, t_3) = \text{foldr}(\lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2; t_1, t_2, t_3).$

$$\frac{\Gamma, x_1 : A, x_2 : \text{Thunk } B \vdash^{\mathbb{L}} t_1 : \mathbb{M} B \quad \Gamma \vdash^{\mathbb{L}} t_2 : \mathbb{M} B \quad \Gamma \vdash^{\mathbb{L}} t_3 : \text{List } A}{\Gamma \vdash^{\mathbb{L}} \text{foldM}(\lambda x_1 x_2. t_1, t_2, t_3) : \mathbb{M} B}$$

To characterize the reduction behavior of L-FOLDM, we define yet another auxiliary relation:

$$\gamma ; \lambda x_1 x_2. t_1, t_2, v \Downarrow^{\mathbb{L}\text{-foldM}} u \equiv \gamma ; \lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2; t_1, t_2, v \Downarrow^{\mathbb{L}\text{-foldr}} u.$$

Lemma 5. *The following rules completely characterize foldM.*

$$\begin{array}{c} \frac{\gamma ; \lambda x_1 x_2. t_1, t_2, v \Downarrow^{\mathbb{L}\text{-foldM}} u}{\gamma ; \text{foldM}(\lambda x_1 x_2. t_1, t_2, v) \Downarrow^{\mathbb{L}} u} \text{ L-FOLDM} \\ \frac{\gamma ; t_2 \Downarrow^{\mathbb{L}} u}{\gamma ; \lambda x_1 x_2. t_1, t_2, \varepsilon \Downarrow^{\mathbb{L}\text{-foldM}} u} \text{ L-FOLDM-NIL} \\ \frac{\gamma, x_1 \mapsto v_1, x_2 \mapsto \perp ; t_1 \Downarrow^{\mathbb{L}} u}{\gamma ; \lambda x_1 x_2. t_1, t_2, (v_1, \perp) \Downarrow^{\mathbb{L}\text{-foldM}} u} \text{ L-FOLDM-UNDEFINED} \\ \frac{\gamma ; \lambda x_1 x_2. t_1, t_2, v_2 \Downarrow^{\mathbb{L}\text{-foldM}} (v_3, c_1) \quad \gamma, x_1 \mapsto v_1, x_2 \mapsto \text{sthunkZ } v_2 ; t_1 \Downarrow^{\mathbb{L}} (v_4, c_2)}{\gamma ; \lambda x_1 x_2. t_1, t_2, (v_1, \text{sthunkZ } v_2) \Downarrow^{\mathbb{L}\text{-foldM}} (v_4, c_1 + c_2)} \text{ L-FOLDM-THUNK} \end{array}$$

Proof. We first prove validity.

- **L-FoldM.**

$$\frac{\gamma ; \lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1, t_2, v \Downarrow^{\mathbb{L}\text{-foldr}} u}{\gamma ; \text{foldr}(\lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1, t_2, v) \Downarrow^{\mathbb{L}} u} \text{ L-FOLDR}$$

- **L-FoldM-Nil.**

$$\frac{\gamma ; t_2 \Downarrow^{\mathbb{L}} u}{\gamma ; \lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1, t_2, \varepsilon \Downarrow^{\mathbb{L}\text{-foldr}} u} \text{ L-FOLDR-NIL}$$

- **L-FoldM-Undefined.**

$$\frac{\frac{\frac{}{\gamma, x_1 \mapsto v_1, y_2 \mapsto \perp ; y_2 \Downarrow^{\mathbb{L}} \perp} \text{ L-VAR}}{\gamma, x_1 \mapsto v_1, y_2 \mapsto \perp ; \text{transpose } y_2 \Downarrow^{\mathbb{L}} (\perp, 0)} \text{ L-TRANPOSE-UNDEFINED} \quad \gamma, x_1 \mapsto v_1, y_2 \mapsto \perp, x_2 \mapsto \perp ; t_1 \Downarrow^{\mathbb{L}} u}{\gamma, x_1 \mapsto v_1, y_2 \mapsto \perp ; \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1 \Downarrow^{\mathbb{L}} u} \text{ L-BIND}$$

$$\frac{\gamma, x_1 \mapsto v_1, y_2 \mapsto \perp ; \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1 \Downarrow^{\mathbb{L}} u}{\gamma ; \lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1, t_2, (v_1, \perp) \Downarrow^{\mathbb{L}\text{-foldr}} u} \text{ L-FOLDR-UNDEFINED}$$

- **L-FoldM-Thunk.** By induction, $\gamma ; \lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1, t_2, v_2 \Downarrow^{\mathbb{L}\text{-foldr}} v_3$. Then,

$$\frac{\frac{\frac{}{\gamma, x_1 \mapsto v_1, y_2 \mapsto \text{sthunkZ}(v_3, c_1) ; y_2 \Downarrow^{\mathbb{L}} \text{sthunkZ}(v_3, c_1)} \text{ L-VAR}}{\gamma, x_1 \mapsto v_1, y_2 \mapsto \text{sthunkZ}(v_3, c_1) ; \text{transpose } y_2 \Downarrow^{\mathbb{L}} (\text{sthunkZ } v_3, c_1)} \text{ L-TRANPOSE-THUNK} \quad \gamma, x_1 \mapsto v_1, y_2 \mapsto \text{sthunkZ}(v_3, c_1)}{\gamma ; \lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1, t_2, v_2 \Downarrow^{\mathbb{L}\text{-foldr}} v_3 \quad \gamma, x_1 \mapsto v_1, y_2 \mapsto \text{sthunkZ}(v_3, c_2) ; \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1 \Downarrow^{\mathbb{L}} (v_4, c_1 + c_2)} \text{ L-FOLDR-THUNK}$$

$$\gamma ; \lambda x_1 y_2. \text{do } x_2 \leftarrow \text{transpose } y_2 ; t_1, t_2, (v_1, \text{sthunkZ } v_2) \Downarrow^{\mathbb{L}\text{-foldr}} (v_4, c_1 + c_2)$$

To prove completeness, repeatedly apply inversion principles to find that any derivation of $\text{foldM}(\lambda x_1 x_2. t_1, t_2, v)u$ must have the form of one of the preceding derivations.

□

$$\lfloor \text{foldr}(\lambda x_1 x_2. t_1, t_2, t_3) \rfloor = \text{do } y_3 \leftarrow \lfloor t_3 \rfloor; \text{foldM}(\lambda x_1 x_2. \lfloor t_1 \rfloor, \lfloor t_2 \rfloor, y_3)$$

5.4.5 Properties of the translation

Lemma 6. *If $\Gamma \vdash^B t : A$, then $\Gamma \vdash^L \lfloor t \rfloor : A$.*

6 Equivalence

6.0.1 Clairvoyance semantics

Theorem 1. $\gamma ; t \Downarrow_c^{\mathbb{C}} v$ if and only if $\gamma ; \lfloor t \rfloor \Downarrow^{\mathbb{L}} (v, c)$.

Proof. ($\implies$) By induction on the derivation of $\gamma ; t \Downarrow_c^{\mathbb{C}} v$.

Case:

$$\frac{\gamma ; t_3 \Downarrow_{c_1}^{\mathbb{C}} v_1 \quad \gamma ; \lambda x_1 x_2. t_1, t_2, v_1 \Downarrow_{c_2}^{\mathbb{C}\text{-foldr}} v_2}{\gamma ; \text{foldr}(\lambda x_1 x_2. t_1, t_2, t_3) \Downarrow_{c_1+c_2}^{\mathbb{C}} v_2} \text{L-FOLDR}$$

- $t = \text{foldr}(\lambda x_1 x_2. t_1, t_2, t_3)$
- $\lfloor t \rfloor = \text{do } y_3 \leftarrow \lfloor t_3 \rfloor; \text{foldM}(\lambda x_1 x_2. \lfloor t_1 \rfloor, \lfloor t_2 \rfloor, y_3)$

We need to construct a derivation of the form

$$\frac{\gamma ; \lfloor t_3 \rfloor \Downarrow^{\mathbb{L}} (v_1, c_1) \quad \gamma, y_3 \mapsto v_1 ; \text{foldM}(\lambda x_1 x_2. \lfloor t_1 \rfloor, \lfloor t_2 \rfloor, y_3) \Downarrow^{\mathbb{L}} (v_2, c_2)}{\gamma ; \text{do } y_3 \leftarrow \lfloor t_3 \rfloor; \text{foldM}(\lambda x_1 x_2. \lfloor t_1 \rfloor, \lfloor t_2 \rfloor, y_3)(v_2, c_2) \Downarrow^{\mathbb{L}} (v_2, c_1 + c_2)} \text{L-BIND}$$

By the inductive hypothesis, we have

$$\gamma ; \lfloor t_3 \rfloor \Downarrow^{\mathbb{L}} (v_1, c_1),$$

so we just need to prove

$$\gamma, y_3 \mapsto v_1 ; \text{foldM}(\lambda x_1 x_2. \lfloor t_1 \rfloor, \lfloor t_2 \rfloor, y_3) \Downarrow^{\mathbb{L}} \lfloor t_2 \rfloor y_3(v_2, c_2).$$

For this, it suffices to prove

$$\gamma ; \lambda x_1 y_2. \text{foldM}(\lambda x_1 x_2. \lfloor t_1 \rfloor, \lfloor t_2 \rfloor, y_3), \lfloor t_2 \rfloor, y_3 \Downarrow^{\mathbb{L}\text{-foldr}} (v_2, c_2).$$

We must also prove the following lemma by mutual induction.

Lemma 7. $\gamma ; \lambda x_1 x_2. t_1, t_2, v_1 \Downarrow_c^{\mathbb{C}\text{-foldr}} v_2$ if and only if $\gamma ; \lambda x_1 x_2. \lfloor t_1 \rfloor, \lfloor t_2 \rfloor, v_1 \Downarrow^{\mathbb{L}\text{-foldM}} (v_2, c)$.

Proof. By induction on the derivation of $\gamma ; \lambda x_1 x_2. t_1, t_2, v_1 \Downarrow_c^{\mathbb{C}\text{-foldr}} v_2$.

- **Case C-Foldr-Nil:**

$$- v_1 = \varepsilon$$

– $\gamma ; t_2 \Downarrow_c^{\mathbb{C}} v_2$

By mutual induction, we obtain $\gamma ; [t_2] \Downarrow^{\mathbb{L}} (v_2, c)$, so we can construct

$$\frac{\gamma ; [t_2] \Downarrow^{\mathbb{L}} (v_2, c)}{\gamma ; \lambda x_1 x_2. [t_1], [t_2], \varepsilon \Downarrow^{\mathbb{L}\text{-foldr}} (v_2, c)} \text{ L-FOLDM-NIL}$$

• **Case L-Foldr-Undefined:**

($\Leftarrow$) By induction on the derivation of $\gamma ; \lambda x_1 x_2. [t_1], [t_2], v_1 \Downarrow^{\mathbb{L}\text{-foldM}} (v_2, c)$. $\square$

$\square$

6.0.2 Demand semantics

$$\begin{aligned} \mathbb{D}[t](x_1 : A_1, \dots, x_n : A_n) = & \text{let } x_1^A = \text{free } A_1 \text{ in} \\ & \text{assert } x_1^A \leq x_1 \text{ in} \\ & \vdots \\ & \text{let } x_n^A = \text{free } A_n \text{ in} \\ & \text{assert } x_n^A \leq x_n \text{ in} \\ & \text{let } c = \text{free Nat in} \\ & \text{assert } \mathbb{C}[t] == (y^A, c) \text{ in} \\ & (x_1^A, \dots, x_n^A, c) \end{aligned}$$

Lemma 8. *Let $\Gamma = x_1 : A_1, \dots, x_n : A_n$. If $\Gamma \vdash^{\mathbf{B}} t : B$ and $\gamma \in \mathbb{E}[\Gamma]$, then $\Gamma, y^A : B \vdash^{\mathbf{L}} \mathbb{D}[t](\Gamma) : M(A_1 * \dots * A_n)$*

Theorem 2. *If $\mathbb{D}[\llbracket t \rrbracket](\gamma, b^A) = (\gamma^A, c)$, then $\gamma, y^A \mapsto b^A ; \mathbb{D}[t](\Gamma) \Downarrow^{\mathbb{L}} (|\gamma^A|, c)$, where $|x_1 \mapsto a_1, \dots, x_n \mapsto a_n| = (a_1, \dots, a_n)$.*

7 Case studies

- various examples showing how our approach works
 - exactly which examples will be based on experimentation: are there examples that Curry has no trouble with? examples that Curry does have trouble with?
 - some candidates:
 - * take from head of a list (discussed earlier)
 - * insertion sort
 - * some amortized data structure
- (including at least one example using an amortized data structure)
- do they work? i.e., are they functionally correct?
- does Curry find all the solutions? does it ever get stuck?
- (maybe) how is the performance? are there any problems?

8 Related works

9 Conclusion